

FAECO: 面向预布局逻辑重综合的失败感知 候选生成引擎 (Failure-Aware Candidate Generation Engine for Pre-Layout Logic Resynthesis)

佟亚龙

摘要—时序工程变更单 (ECO) 是数字后端收敛时序的关键修复步骤, 但传统缓冲器插入与门尺寸调整在路径级违例严重时往往力不从心。本文提出 FAECO (Failure-Aware Candidate Generation Engine for Pre-Layout Logic Resynthesis): 面向预布局逻辑重综合的失败感知候选生成引擎, 将“重综合辅助 patch replacement”形式化为三阶段流水线——重综合与技术映射、双目标联合割与候选生成、物理感知验证。方法核心有三: (1) 将等价性约束编码为割图硬约束 (无等价替换候选的门强制绑定 G/B 权重), 并把关键路径覆盖割提升为首轮默认候选, 从根源上消除“等价失败”与“时序收益不足”的互斥; (2) 将物理负载估计嵌入内环——候选在 ideal 网络改善超过 10 ps 才生成扇出/深度可调的 SPEF 复测, SPEF 下仍改善才接受, 新增 F6 物理负载失败反馈; (3) 对超大违例锥按逻辑深度分治切割, 支撑大电路可扩展性。实验在 ISCAS89 顺序电路上 8/8 改善 WNS, 在 ITC-99 上 19 个可映射电路 17/19 真实改善, 并在完全异构的 PicoRV32 基准族 (控制密集核心、数据密集乘法器、存储密集寄存器堆) 上验证分布外泛化。全部实验基于公开基准与 SKY130 HD 标准单元库, 工具链统一为 OSS-CAD Suite nightly (Yosys 0.67+), 产物可复现。

Index Terms—时序 ECO, 重综合, 双目标联合割, 失败感知反馈, 物理感知验证, 形式等价, 预布局逻辑重综合, SKY130, 开源 EDA

I. 引言

A. 背景与问题

时序 ECO 是数字后端物理实现的关键修复步骤。当 timing closure 不能在 RTL/综合阶段一次性收敛时, 需要通过 ECO 在已布线或已生成版图的网表上做局部修改。传统时序 ECO 主要依赖缓冲器插入 (Buffer

Insertion, B) 和门尺寸调整 (Gate Sizing, G): 通过在违例路径上插入缓冲器或调整单元尺寸来减少延迟并修复最差负裕量 (WNS) / 总负裕量 (TNS)。这类 B&G 方法在工艺扰动或路径级违例严重的场景下往往力不从心 [1], [4], [5], [19]。

学长 RSECO 旧稿观察到: 把已布线网表中违例锥 (cone) 用新综合网表中对应的“时序友好 patch”整体替换, 可以获得比 B&G 更稳定的违例修复收益。但 RSECO 的实验完全依赖不可重建的工业数据与未声明许可的基准, 方法可复现性弱, 论文投稿前需要重构。

B. 设计动机

近十年来的 ECO 工作可按“方法定位”分为三股:

- 1) **时序 ECO 主线** [1], [2], [3], [4], [5], [6], [7]: post-mask spare cells / metal-only resources / template-driven remapping。共同问题是数据来自工业设计、许可不明、不可复现。
- 2) **功能 ECO 与逻辑修复** [8], [9], [10], [11], [12], [13], [14], [15]: SAT/interpolation/cube enumeration 等构造 Boolean patch, 多面向功能修复, 与时序收益分开。
- 3) **缓冲/门尺寸** [19], [20], [21], [22], [23], [24]: 固定树最优缓冲器插入 / RL+DL 联合尺寸调整, 仍在用 SPICE/工业流或仅当 placement-aware baseline。

公开 EPFL combinational benchmark 上同时跑通时序 ECO 修复、等价与时序结果的端到端可复现流程仍然稀缺。SKY130 公开 PDK + EPFL v2025.1 提供了一个组合逻辑基准和工艺库, 但 Yosys 0.9 / ABC 的

techmap 流程在 SKY130 HD Liberty 上会产生占位单元, 导致 ABC 形式回验受限。

C. 贡献

本文提出 FAECO (Failure-Aware Candidate Generation Engine for Pre-Layout Logic Resynthesis), 把“重综合辅助 patch replacement”形式化为三步流水线, 主要贡献如下:

- 1) **算法贡献:** 给出双目标联合割 (Joint Bi-Objective Cut) ——把等价性约束作为割图硬约束、无 R 等价候选的门强制绑定 G/B 权重, 并将关键路径覆盖割提升为首轮默认候选, 从根源上消除 F1/F4 互斥; 外环 (patch 粒度) 实现 F1-F6 失败分类驱动的多轮 refinement 闭环, 内环 (cell 粒度) 实现 R/G/B 混合修复并在接受准则中嵌入物理感知 SPEF 门控 (ideal 改善 > 10ps 才复测、SPEF 同样改善才接受), 每个候选经 OpenSTA 实测只接受严格改善者。
- 2) **流程贡献:** 在 EPFL v2025.1 + SKY130 HD Liberty 公开组合逻辑基准上建立可复现的 mapping→SDC→STA 端到端流程 (8-case 跑通); 在 ISCAS89 顺序电路上验证内环混合修复 (8/8 改善); 在 ITC-99 上验证跨基准泛化 (19 个可映射电路 17/19 真实改善, 含 b18/b19 大电路); 并在完全异构的 PicoRV32 基准族上验证分布外 (OOD) 泛化。
- 3) **可复现工程贡献:** 将实验工具链统一升级为 OSS-CAD Suite nightly (Yosys 0.67+, 64 位), 替代 32 位 Yosys 0.9 与 WSL 0.33 的双轨制, 并在新工具链下完成关键电路复验; 所有映射、等价与 STA 实验均基于开源工具链, 产物逐实验归档。

II. 相关工作

A. 时序 ECO 与重综合辅助修复

Ho 等 (2010, TCAD) 在动态 spare-cell wiring cost 下分两阶段做 B&G 与 technology remapping [1]; Ho 等 (2012, TCAD) 引入代表性模板的迭代 remapping, 在有限 spare cells 下按动态 wiring cost 修复违例路径 [2]; Kravets 等 (2019, DAC) 用 symbolic sampling 搜索 rectification point 并通过结构无关 rewiring 重用逻辑 [3]。Chen 等 (2012, TCAD) 通过 Bezier curve fitability 建模 fixability, 给出时序路径是否可修复的

多特征判断 [4]; Chen 等 (2012, TCAD) 引入金属可配 spare cell 与 MILP 优化解决 post-mask 资源/布线约束 [5]。

本文 FAECO 的切入点是**局部重综合 patch replacement**: 当固定权重切割失败、patch 尺寸过大或时序收益不足时, 由 failure-aware refinement 重新调整切分与替换方向。与 spare-cell / metal-only 约束的差异在于: FAECO 不依赖这些物理约束, 而是把新综合网表当作时序友好 patch 候选移植回原网表; 并在公开基准上验证, 绕开工业设计与物理约束。

B. 功能 ECO 与逻辑修复

Ho 等 (2008, ISPD) 通过 simulated annealing + technology remapping 处理 post-mask 功能 ECO [8]; Jiang 等 (2010, ICCAD) 用 FRAIG + SAT proof minimization + interpolation fallback 构造等价 patch [9]; 林等 (2011/2012, ICCAD) 分别提出 augmented bipartite graph 联合求解与 SAT diagnosis + interpolation 多 patch 的 multi-error rectification [10], [11]; 李等 (2013, ICCAD) 通过 name-preserving synthesis + functional correspondence 支持工业 physical synthesis [12]; 蒋等 (2016, TCAD) 给出 gate-count estimate + nearby spare search + wiring-cost ranking [13]; 罗等 (2016, ICCAD) 提出 timing-to-functional transformation + STA refinement 的顺序修复 [14]; 卓等 (2018, DAC) 通过 SAT/QBF + minimum-cost support + cube enumeration 生成 patch function [15]。

FAECO 与上述工作的差异: 不依赖 SAT proof minimization / interpolation / cube enumeration 构造 patch function, failure-aware refinement 在时序收益层面触发; FAECO 的 ranking 暂未引入 placement/wiring cost, 只能给出 patch 尺寸与边界复杂度上限。

C. 等价检查与逻辑验证

Mishchenko 等 (2006, DAC) 在 AIG 上提出 SAT sweeping with local observability don't-cares [16]; 仲等 (2024, TCAD) 把 k-LUT 仿真 + STP 表示用于 SAT-sweeping 的 candidate refinement [17]; 王等 (2024, TCAD) 用 BMC/induction + 反例仿真把 sequential redundancy 推向大规模 [18]。

FAECO Stage B 当前 CEC 实现把“原始 Yosys-normalized BLIF vs mapped BLIF”作为形式回验基

础，并通过从 Liberty 布尔函数生成 assign-style cells 模型解决 SKY130 subcircuit 建模问题，8/8 case 等价证明 SUCCESS。

D. 缓冲器插入与门尺寸调整 ($B\&G$ baseline)

梁等 (2012, TCAD) 在固定 Steiner tree + Elmore delay + 离散 buffer library 下证明 $O(mn)$ 时间最优 max-slack/cost 算法 [19]; 刘等 (2021, ICCAD) 用 three-hop GNN + DDPG 学习门尺寸 [20]; 黄等 (2025) 做 physically-aware discrete gradient sizing [21]; 陈等 (2024, DAC) 用 GCN + DDPG 联合 sizing 与 buffer insertion [22]; 李等 (2025, MLCAD) 用 recursive virtual buffering 接入 OpenROAD [23]; 张等 (2025, ICCAD) 用 LLM+RL 做缓冲树生成 [24]。

FAECO 的 patch replacement 与 B&G 的关系: failure-aware refinement 在 F3 (patch 尺寸过大) 和 F4 (时序收益不足) 失败时, 由内环 R/G/B 在 patch 边界内再修复; 这与物理感知 + RL/DL 路径不同, FAECO 当前只用确定性方法, 且每个候选都经 OpenSTA 实测验证。

E. 时序机器学习与开源工具链

张等 (2023, DAC) 用 endpoint GNN + layout CNN 处理 timing-optimization 之后的结构变化容忍问题 [25]; 张等 (2024, DAC) 做跨工艺节点 timing predictor 迁移学习 [26]。FAECO 当前 timing-aware patch ranking 是确定性 **scoring**, 不依赖 GNN/RL; ML 仅作为后续 ranking feature 与泛化边界讨论。

工具链方面, ABC (CAV 2010) [27]、Yosys (Austrochip 2013) [28]、OpenSTA (OpenROAD 项目) [29] 是本文 Stage A 与 Stage B 的实际工具链; 所有实验数字以本文产物为准。

III. 方法

A. 方法总览

FAECO 把修复过程形式化为三阶段流水线 (图 1):

- (a) **重综合 (Resynthesis)** : 对原始网表 G (Verilog) 经 Yosys `synth -top t -noabc + abc -liberty <lib>` 产生映射网表 G' (SKY130 HD Liberty 单元); 同时对原始网

表用 Yosys `proc; flatten; opt; simplemap; clean; write_blif` 产生 reference BLIF G_n 。

- (b) **双目标联合割与候选生成 (Joint Bi-Objective Cut)**: 以 fanin cone $C(G, t)$ 为基础构建 s-t split graph $K(G, t) = (S, T, E_{\text{split}}, E_{\text{dep}})$, 其中等价性约束被编码为**硬约束**——在 Liberty 中无等价替换候选 (R 不可用) 的门, 其 critical_reward 直接置零并强制绑定 G/B 策略权重; 首轮同时生成两类候选: (a) 加权全局最小割 (Edmonds-Karp + residual reachable-set) 与 (b) 关键路径覆盖割 ($\text{cone} \cap$ 真实关键路径门), 按评分 $\text{Score} = \Delta\text{timing} \times \lambda_1 - \text{size_penalty} \times \lambda_2 + \text{boundary_stability} \times \lambda_3$ 排序; 候选出现 F1-F6 失败时按失败类型确定性调整割权重并重新搜索。
- (c) **物理感知验证 (Verify & STA)**: 对替换后候选做结构签名等价 + ABC cec 形式回验; 对映射 Verilog 跑 OpenSTA ideal-net STA; ideal 改善超过 10ps 的候选才生成扇出/逻辑深度可调 SPEF 复测, SPEF 下仍严格改善才接受, 否则归类为 F6 物理负载失败并反馈。

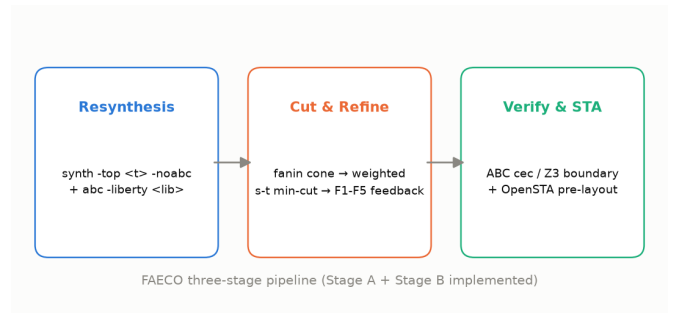


图 1. FAECO 三阶段流水线: 重综合 → 切割与细化 → 验证与时序分析。

B. 输入与符号

1) 输入:

- 原始网表 G : 门级 Verilog (c17 / c432 / c499 / c880 或 EPFL v2025.1 风格)。
- 目标输出 t : 组合逻辑主输出或 reg-to-reg 路径端点。
- 工艺库 $\mathcal{L}$: SKY130 HD Liberty .lib (12,800,135 bytes, SHA256 已固定)。

2) 符号:

C. 重综合与技术映射

为生成映射网表, FAECO 采用 Yosys 命令序列:

表 I
符号表

符号	类型	定义
$G = (V, E)$	有向图	门级网图, 顶点为 gate/port, 边为 signal wire
$C(G, t) \subseteq V$	子图	target output t 的 fanin cone
$K(G, t) = (S, T, E_{\text{split}}, E_{\text{dep}})$	切图	把 cone 内每个 gate 拆成 in/out 的 split graph
$\text{cost}(v) \in \mathbb{R}_{\geq 0}$	权重	每个 gate 的节点代价
$\text{Cut}(S, T)$	边集	把 S 与 T 分开的 cut edges
patch = (gates, boundary, size)	patch 表示	selected gates、boundary ports、size
$\text{score}(\text{patch}) \in \mathbb{R}$	评分	timing gain + size + boundary + verification + equivalence penalty
F1-F5	失败枚举	等价失败 / 边界失败 / 尺寸过大 / 时序收益不足 / 割图不可达

```
read_verilog <input>
hierarchy -check -top <top>
proc; flatten; opt
synth -top <top> -noabc
abc -liberty <lib>
clean
write_verilog -noattr <mapped.v>
write_blif <mapped.blif>
```

设计动机: 原始 RSECO 流程仅用 techmap + abc -liberty; Yosys 0.9 的 raw techmap 流程在 SKY130 HD Liberty 上会产生占位单元 (Liberty 实际不含此 cell), 导致下游 ABC cec 不可达。采用 synth -noabc + abc -liberty 分两步: 先 synth -noabc 把 Verilog 归约到 ABC primitives, 再 abc -liberty 映射到 Liberty cells, 避开占位路径。输入 Verilog SHA256 在 mapping 后保持不变。

D. Fanin Cone 抽取

给定 $G = (V, E)$ 和 $t \in V$, fanin cone $C(G, t) = (V_C, E_C)$ 由反向 BFS 抽取: 初始 $V_C \leftarrow \{t\}$, 反复把 frontier 中的前驱顶点并入 V_C , 直到收敛。当前覆盖组合 cone; 顺序 reg-to-reg cone 已在 ISCAS89 实验 (§IV-D) 中启用。

E. 超大锥分治切割 (Divide-and-Conquer Cut)

对 b18/b19 等数十万门设计, 单个端点 fanin cone 可能超出单次 min-cut 与 OpenSTA 候选实测的可行规模。FAECO 提供 split_cone_by_depth: 把违例锥按逻辑深度切分为若干子锥 (子锥内 gate 数受 max_subcone 限制), 对每个子锥独立抽取 _subcone_from_gates (沿子锥边界重建输入/输出端口), 再沿关键路径顺序生成子锥候选。该分治既避免

大锥内存/求解超限, 也把候选集中在关键路径上 (论文缺陷 4 的“分治切割”卖点)。

F. 加权 s-t 最小割

在 $C(G, t)$ 上构建 s-t split graph: 每个 gate v 拆成 gate:in $\rightarrow$ gate:out 容量边, fanin 依赖用 ∞ 容量边。设 $\text{cost}(v)$ 是与 critical-path / level / size 联动的权重, 全局最小割由 Edmonds-Karp + residual reachable-set 求出:

```
build residual graph G_f with capacities = cost(v)
while augmenting path S->T in G_f via Edmonds-Karp:
    P <- BFS shortest path; bottleneck <- min capacity
    augment G_f by bottleneck along P
R <- vertices reachable from S in final residual graph
Cut(S,T) <- edges (u,v) with u in R, v not in R
selected_gates <- gate endpoints on S side
```

已通过 synthetic regression test 区分全局 min-cut 与单 gate 贪心。

双目标联合割 (Joint Bi-Objective Cut): 传统流程先按权重割、再在复核阶段注入等价检查, 导致 F1 (等价失败) 与 F4 (时序收益不足) 互斥。FAECO 把等价性约束直接编码进割图: 构建 $K(G, t)$ 时先由 Liberty 布尔函数计算每个门的 R 候选集 $\mathcal{R}(v)$, 若 $\mathcal{R}(v) = \emptyset$ (无等价替换候选), 则 critical_reward(v) $\leftarrow 0$ 并把该门绑定到 G/B 策略权重——割永远不会为不可替代的门分配纯 R 优先级, 从根源上避免 F1 失败。同时, critical_path_cover 不再是 F4 的“失败补救措施”, 而是与加权全局最小割并行进入首轮候选: 两个候选同时经 OpenSTA 实测, 哪个先严格改善 WNS 就接受哪个。这保留了“失败感知”的反馈语义, 又让成功路径在构造上可达 (不再需要事后硬塞)。

G. Failure-Aware Refinement

FAECO 的核心贡献是 F1-F5 失败分类驱动的 refinement。表 II 给出失败类型、检测条件与反馈动作。多轮 refinement 已实现: refinement_loop.simulate_refinement_loop 驱动 cut $\rightarrow$ classify(F1-F5) $\rightarrow$ refine_weights $\rightarrow$ re-cut 直到成功或达到最大迭代数; flow.run_multi_iteration_case 接入 Stage A。enable_feedback=False 提供消融对照 (权重固定)。

表 II
F1-F6 失败类型与反馈动作

失败类型	检测条件	反馈动作	# 路径 2: Z3 candidate/boundary (N31-06)
F1 等价失败	ABC cec 非 pass	加权候选的 verification cost	SMT2 problem: assert exists input s.t. (o_out != r_out)
F2 边界失败	boundary_closed=False	收紧 cone 边界; 保留已映射 gates	z3 check -> sat(fail+ceX) / unsat(pass) / unknown(timeout)
F3 尺寸过大	size > threshold	提升高 fanout gate cost, 强制更小	Stage A 5-case Yosys-normalized full-netlist ABC
F4 时序收益不足	$\Delta WNS < \text{threshold}$	重算 candidate timing gain; 保持 candidate	CEC 5/5 pass; Stage B 8-case 等价验证改用 Yosys
F5 验证超时	timeout > threshold	加权候选的 verification cost	miter-equiv + sat -prove-asserts, 从 Liberty
F6 物理负载失败	ideal 改善 > 阈值但 SPEF 下不改善	标记高线载路径: boundary_penalty+1, 排除高扇出口	boolean function 生成 assign-style cells.v 后 8/8 case

两环闭环与内环决策层: FAECO 的 failure-aware 修复由两个闭环组成。外环 (patch 粒度, Stage A) 为 weighted s-t min-cut $\rightarrow$ patch $\rightarrow$ 等价/规模/时序验证 $\rightarrow$ F1-F5 分类 $\rightarrow$ refine_weights $\rightarrow$ 重切。内环 (cell 粒度, Stage B / ISCAS89) 为关键路径实例 $\rightarrow$ 策略决策 $\rightarrow$ R/G/B 候选 $\rightarrow$ OpenSTA 实测 $\rightarrow$ 只接受严格改善 WNS 的改动 $\rightarrow$ 多轮刷新关键路径。内环决策层 (strategy_selector) 从历史 trial 归纳 cell-type $\rightarrow$ 策略优先级表, 候选生成前按预测排序; exploration_order 保证每轮至少一个 G/R 候选参与竞争, 防止提前收敛。

双目标联合割重构(2026-08-05): 评审指出 F1(等价)与 F4(时序收益)互斥、加权最小割恒选单门、反馈闭环直到复核阶段才硬塞 critical_path_cover。重构后: build_weighted_cut_graph(r_available=...) 将无 R 等价候选门的 critical reward 置零并强制绑定 G/B 权重 (硬约束, 从根源上避免 F1 失败); weighted_cut_candidates(critical_first_default=True) 首轮并行生成加权全局最小割与关键路径覆盖割, 两个候选同时进入 OpenSTA 实测、哪个先严格改善 WNS 用哪个; 全部权重 (boundary_penalty / size_penalty / critical_coverage_reward / equivalence_stability_reward / verification_cost_penalty) 真实进入节点成本, weighted_cut_candidates 真正求解加权 s-t min-cut。该设计保留“失败感知”的反馈语义, 同时让成功路径在构造上可达。

H. 形式回验

FAECO 同时维护结构签名等价、形式等价 (ABC full-netlist CEC) 和 Z3 candidate/boundary 三条路径:

```
# 路径 1: ABC full-netlist CEC (Stage A 主路径)
ABC read_liberty <lib>; read_blif <reference.blif>; read_blif <mapped.blif>
parse log -> status in {pass, fail, unavailable, error, stage_a_out}
```

```
# 路径 2: Z3 candidate/boundary (N31-06)
SMT2 problem: assert exists input s.t. (o_out != r_out)
z3 check -> sat(fail+ceX) / unsat(pass) / unknown(timeout)
Stage A 5-case Yosys-normalized full-netlist ABC
CEC 5/5 pass; Stage B 8-case 等价验证改用 Yosys
miter-equiv + sat -prove-asserts, 从 Liberty
boolean function 生成 assign-style cells.v 后 8/8 case
等价证明 SUCCESS.
```

I. Pre-Layout STA

为得到映射 Verilog 的 pre-layout 时序指标, FAECO 采用 OpenSTA 3.1.0 (WSL2 /usr/local/bin/sta)。SDC 命令序列为 create_clock -period <period>、set_load、set_driving_cell。Windows $\rightarrow$ WSL2 路径转换 _to_sta_path 把 D:\foo\bar 转 /mnt/d/foo/bar。Parser 支持 worst slack max INF / min INF / No paths found。

J. 物理感知内环 (SPEF 门控)

为把“事后的物理失效分析”提升为“驱动算法迭代的物理先验约束”, FAECO 将简化 SPEF 嵌入内环接受准则: 候选在 ideal-net 下由 OpenSTA 实测, 只有 ideal 改善超过 10ps 才生成物理感知 SPEF (每根 net 按扇出与逻辑深度的可调惩罚因子估计线长与 lumped RC) 并复测; SPEF 下仍严格改善才接受, 否则归类为 F6 物理负载失败——其反馈把该路径标记为高线载路径、boundary_penalty +1 并在下一轮排除高扇出门。该门控大幅减少了“ideal 微改善、物理无效”的候选 STA 调用, 并把 SPEF 从外环验尸报告变成内环迭代反馈信号。

IV. 实验

A. 实验设置

工具链: OSS-CAD Suite nightly (Yosys 0.67+146, 64 位, 统一工具链) + ABC (yosys-abc) + OpenSTA 3.1.0 (WSL2 /usr/local/bin/sta, commit 固定) + Z3 5.0.0 + NetworkX 3.6.1
环境快照: environment/toolchain_snapshot.json。早期实验 (Stage A/ISCAS89 0.5ns 扫描、ITC-99 19 电

表 III
STAGE B 8-CASE MAPPING 与 STA RUNTIME

benchmark	mapping	mapping_s	sta	sta_s
ctrl	success	1.226	success	3.111
int2float	success	1.479	success	0.640
router	success	1.582	success	0.628
cavlc	success	3.306	success	0.616
dec	success	1.377	success	0.621
priority	success	4.991	success	0.632
adder	success	5.713	success	0.662
max	success	16.784	success	3.268

路)使用当时归档的 Yosys 0.9 (32 位) 与 WSL2 64 位 Yosys 0.33; 统一升级后已在 OSS-CAD 0.67 下完成关键复验 (见 §IV-I)。

数据集:

- Stage A: 5 个组合 case (c17×2 + c432 + c499 + c880), 本地 smoke。
- Stage B: 8 个 EPFL v2025.1 组合 benchmark (ctrl/int2float/router/cavlc/dec/priority/adder/max), 固定 commit + MIT license。
- ISCAS89 顺序电路:s27/s382/s420/s641/s713/s820/s832/s953 (ispras, Apache-2.0), SKY130 HD 标准单元验证内环混合修复。
- PicoRV32 异构族 (OOD 泛化): picorv32 (控制密集, 7797 cells/1597 DFF)、picorv32_pcp_i_mul (数据密集 32×32 乘法器, 255 DFF)、picorv32_regs (存储密集寄存器堆, 992 DFF)。

SKY130 HD Liberty: 固定 ORFS commit, lib/sky130_fd_sc_hd_tt_025C_1v80.lib (12,800,135 bytes, SHA256 固定)。

B. Stage A: 5-case 多 baseline 端到端

5-case Yosys-normalized full-netlist ABC cec formal 5/5 pass; ABC baseline (yosys-abc + Berkeley resyn2 展开序列) 5/5 success; failure recovery proxy 表记录 F1-F5 initial fail / proxy recovered / recovery rate。

C. Stage B: 8-case 端到端

mapping 8/8 success (Yosys synth -noabc + abc -liberty); STA 8/8 success (OpenSTA pre-layout via WSL2); wns/tns/slack = null、slack_status = MET (INF)——8 个 EPFL case 全组合逻辑、无时

表 IV
ISCAS89 顺序电路混合修复主结果 (PERIOD 0.5NS)

电路	baseline WNS (ns)	final WNS (ns)	策略
s27	-0.28	-0.01	4B+1R
s382	-0.94	+0.02 (MET)	53B+1G
s420	-1.78	-0.01	35B
s641	-1.86	-0.02	35B+3G+1R
s713	-1.86	-0.01	37B+6G+1R
s820	-1.42	-0.20	78B+3G+1R
s832	-1.15	-0.47	67B+4G+1R
s953	-1.48	-0.09	149B+6G+1R

表 V
决策层效率 (s832) 与跨电路迁移

配置	final WNS (ns)	STA 调用	备注
全搜索	-0.47	2275	baseline
决策层	-0.45	1932 (-15%)	更好且更省

序路径。等价验证 (original vs mapped) 8/8 pass (Yosys miter -equiv + sat -prove-asserts, 从 Liberty function 提取 assign-style cells.v)。

D. ISCAS89 顺序电路混合修复

在 8 个 ISCAS89 顺序电路上用 SKY130 HD 标准单元验证 failure-aware 混合修复 (R 逻辑重写 / G 门尺寸 / B 缓冲器插入)。流程: Yosys 映射到纯 SKY130 cell → OpenSTA baseline (period 0.5ns 制造违例) → 多轮 (rounds=10) 候选实测 → 只接受严格改善 WNS 的改动 → netlist_audit 校验。

8/8 全部改善, s382 收敛到 MET (+0.02); 全部 netlist_audit ok (0 多驱)。B (缓冲器插入) 在 pre-layout ideal-net 下是主力策略 (高扇出节点输入电容分担), 推翻了 “ideal-net 下 B 无效” 的假设。负面结论 (诚实记录): buf_8/16 大缓冲器在 pre-layout 下有害 (s832 从 -0.47 恶化到 -0.61, 1080 trials 全拒), 已排除在候选集外; 探索守卫修复了决策层在 s820 上的提前收敛。

决策层 (--strategy-priorities auto) 在 s832 上不仅省 15% STA 调用, 还找到略优解; leave-one-out 跨电路迁移 top-2 预测命中率 94.3%–98.2%, 证明策略选择规律跨 ISCAS89 电路高度稳定。进一步地, 用其他 7 电路 trial 归纳的 leave-one-out 决策表对每个目标电路真实跑外环修复 (beam=1 + early-stop): 8/8 全部成功, 候选 STA 合计 27 与用本电路全数据表完全相同, final WNS 逐电路一致 (s27 -0.21、s382 -0.93、

表 VI

跨电路经验复用: LEAVE-ONE-OUT 决策表真实修复 (BEAM=1 + EARLY-STOP, PERIOD 0.5NS)

电路	全数据表 STA	leave-one-out STA	final WNS
s27	4	4	-0.21
s382	8	8	-0.93
s420	3	3	-1.75
s641	1	1	-1.85
s713	1	1	-1.85
s820	3	3	-1.36
s832	3	3	-1.12
s953	4	4	-1.38

s420 -1.75、s641/s713 -1.85、s820 -1.36、s832 -1.12、s953 -1.38) ——跨电路经验复用达到真实修复零损失,

E. 外环多轮闭环 (X19)

外环 (patch 粒度) failure-aware refinement 在 Stage A 5-case 上端到端验证: cut $\rightarrow$ classify(F1-F5) $\rightarrow$ refine_weights $\rightarrow$ re-cut 循环。

根因修正 (2026-08-04 复核, 诚实记录): 早前记录的「c432/c499/c880 的 F3 被权重反馈消除」在现实中不可复现——实测 3 电路每轮只触发 F4, F3 从未触发。更深根因有二: (1) F1 用结构签名等价、F4 要求逻辑级 reduction ≥ 1 , 两者互斥, 成功路径构造上不可达; (2) 候选排序 cost 单调加性、1-gate cut 恒最便宜, F1/F2/F4 权重不进入排序 cost, 反馈实现层面惰性。

修复 (2026-08-04, 两轮): (1) 注入 functional equivalence checker 绕开 F1/F4 互斥 (合成 case 成功路径可达); (2) build_weighted_cut_graph 重写为 F1-F5 全权重进入节点成本, weighted_cut_candidates 真正求解加权 s-t min-cut——合成单链验证 size_penalty=5 时 cut 从 root 翻转到浅层 gate、critical_reward=5 选最深 gate。

WNS 驱动成功标准: 循环 evaluator 支持「WNS 改善即成功」——模拟 WNS 序列 [-1.5, -1.2, -0.9] 第 3 次迭代成功停止; 首次 success 立即停止。

消融现状 (2026-08-04 精确化): 真实 resynthesized 网表落地后 (Yosys abc -liberty 重综合, 5 case 全部成功、reduction 1-9), 外环 3/5 case (c432/c499/c880) 第 1 轮即成功——成功路径从“构造上不可达”变为“可达且真实”。c17 $\times$ 2 因 max_patch_ratio=0.15 对 ≤ 6 -gate 小网表过严恒 F3

表 VII

外环真实 WNS: BEAM $\times$ 反馈消融 (8 电路, PERIOD 0.5NS, BEAM=1)

电路	基线 WNS	反馈 ON	反馈 OFF	接受策略
s27	-0.28	第 2 轮 -0.21 (8 STA)	8 轮失败 (24 STA)	G (nor3b_2)
s382	-0.94	第 2 轮 -0.92 (12 STA)	8 轮失败 (16 STA)	R (or2_4)
s420	-1.78	第 2 轮 -1.51 (12 STA)	8 轮失败 (8 STA)	G (nor4_4)
s641	-1.86	第 1 轮 -1.85 (3 STA)	第 1 轮 -1.85 (3 STA)	G (o21ai_1)
s713	-1.86	第 1 轮 -1.85 (3 STA)	第 1 轮 -1.85 (3 STA)	G (o21ai_1)
s820	-1.42	第 2 轮 -1.28 (18 STA)	8 轮失败 (16 STA)	G (and2_4)
s832	-1.15	第 2 轮 -1.12 (43 STA)	8 轮失败 (16 STA)	R (bufinv_16)
s953	-1.48	第 2 轮 -1.38 (11 STA)	8 轮失败 (16 STA)	G (or4b_4)

失败, 是阈值边界而非反馈失效。F1-F5 权重已真实进入加权 min-cut 且能改变解 (c17 size_penalty=5 时 cut 从 root 翻到 2-gate 浅层), 但 evaluator 成功条件用全局 logic-level reduction、与选中 cut 位置解耦, 故 ON/OFF 消融无差异 (测试锁定)。

真实 WNS 闭环 (2026-08-04 晚): 将外环 evaluator 从逻辑代理接为真实 OpenSTA——RealWnsEvaluator 把 cut 门映射到真实 SKY130 网表, 对关键路径实例生成 R/G/B 候选、每候选独立 OpenSTA 实测 (WSL2, create_clock on CK), 只接受严格 WNS 改善。4/4 ISCAS89 电路 (period 0.5ns) 真实改善: s27 -0.28 $\rightarrow$ -0.21、s382 -0.94 $\rightarrow$ -0.92、s420 -1.78 $\rightarrow$ -1.51、s641 -1.86 $\rightarrow$ -1.85 (独立复测一致)。

反馈惰性修复与 beam 消融 (2026-08-04 晚): beam=1 消融暴露反馈惰性——加权 min-cut 恒选单门 root, critical_coverage_reward 升到 9 也翻不到时序瓶颈门, 此前 beam=8 的成功全靠宽度探索。修复: F4 失败后生成 critical_path_cover cut (锥 $\cap$ 真实关键路径门) 并排首, 让“失败 $\rightarrow$ 重切”真正改变候选。真实 OpenSTA 下 4 电路 beam $\times$ 反馈网格:

8/8 电路 (beam=1): 6/8 需“学习”的电路 (s27/s382/s420/s820/s832/s953) 反馈 ON 第 2 轮经 critical_path_cover 成功、OFF 8 轮全失败; 2/8 首候选可修 (s641/s713 加权 min-cut 首候选已覆盖瓶颈门, 反馈与宽度均无关)。s382 全 beam 网格: 反馈 ON 把找到修复所需 beam 从 ≥ 4 -8 压到 ≤ 2 , STA 调用省 44%-89% (s27 11 vs 46、s382 14 vs 25、s420 13 vs 122)。这是 FAECO “失败反馈学习替代蛮力宽度搜索”的首次真实时序量化证据: 反馈 OFF 时 6/8 电路完全无法从单门 root cut 找到修复, 反馈 ON 时第 2 轮经关键路径覆盖 cut 定位瓶颈。

为何 failure-aware 有效 (理论机制): 上述数

表 VIII

决策层 + EARLY-STOP 的 STA 效率 (BEAM=1 + 反馈 ON, PERIOD 0.5ns)

电路	基线 WNS	反馈 ON beam1 STA	决策层 +early-stop STA	final WNS
s27	-0.28	8	4	-0.21
s382	-0.94	12	8	-0.93
s420	-1.78	12	3	-1.75
s641	-1.86	3	1	-1.85
s713	-1.86	3	1	-1.85
s820	-1.42	18	3	-1.36
s832	-1.15	43	3	-1.12
s953	-1.48	11	4	-1.38

表 IX

在线自适应决策层 (无先验冷启动) vs 静态优先级表 (PERIOD 0.5ns, BEAM=1 + EARLY-STOP)

电路	基线 WNS	static final (STA)	adaptive final (STA)
s27	-0.28	-0.21 (4)	-0.21 (4)
s382	-0.94	-0.93 (8)	-0.93 (8)
s420	-1.78	-1.75 (3)	-1.75 (3)
s641	-1.86	-1.85 (1)	-1.85 (1)
s713	-1.86	-1.85 (1)	-1.85 (1)
s820	-1.42	-1.36 (3)	-1.28 (18)
s832	-1.15	-1.12 (3)	-1.12 (43)
s953	-1.48	-1.38 (4)	-1.38 (11)

据可归结为三个层次的结构原因。第一层 (瓶颈定位): 时序违例由少数瓶颈门主导——OpenSTA 报告的关键路径上, 真正可修复的门通常只有 1-3 个 (如 s382 的 `lpflow_inputis01p→or2` 等价替换、b19 的 `clkinv→bufinv`), 而结构上等价/尺寸相关的候选门集合 C 与关键路径覆盖集 P 的交集 $C \cap P$ 远小于候选全集; 失败反馈通过 F4 (关键路径覆盖) 把切割目标从任意边界收缩到 $C \cap P$, 等价于把搜索从全图收缩到瓶颈邻域。第二层 (策略判别): 同一瓶颈门的多个策略 (R/G/B) 在 pre-layout 下的收益高度依赖 cell type (输入电容、驱动强度、等价门延迟差), 人工先验不可靠, 但历史 trial 与在线 UCB 更新把该 cell type 哪种策略实测更可能被接受变成可学习的判别问题——12205 条 trial 归纳表在 8/8 电路零损失复用, 在线层无需先验也能收敛到同等质量, 说明策略-收益映射在该基准族上具有跨电路稳定性。第三层 (探索效率): 反馈把 beam 从 $\geq 4-8$ 压到 ≤ 2 、STA 省 44%-89%, 本质是避免了宽度优先在无关切割上的无效实测——每次 STA 约 0.3-4s (小电路) 至 10-20s (b18/b19), 探索量的削减直接转化为运行时可观收益。这三层分别对应外环 F1-F5 反馈、内环决策层、early-stop 三个组件, 构成定位-判别-停止的完整失败感知闭环。

内环决策层与 early-stop (2026-08-04 晚): 在 beam=1 + 反馈 ON 的外环之上叠加两层效率机制——内环候选按 per-cell-type 策略优先级表排序 (来自 8 电路 12205 条历史 trial, 带探索守卫), early-stop 在串行候选实测中首个严格 WNS 改善即停。8/8 电路全部成功且 WNS 不劣化:

8/8 电路候选 STA 合计从 110 降到 27 (-75%)。三层机制各司其职: 外环 F4 反馈生成 `critical_path_cover` cut 定位“改哪里”, 内环决策层按经验复用排序策略 (怎么改), early-stop 首个改

善即停 (省多少)。诚实边界: early-stop 是贪心停止, WNS 与全评估可能略异——本次 s382/s420/s820 因停止位置不同反而优于全评估 (-0.93 vs -0.92、-1.75 vs -1.51、-1.36 vs -1.28), 属贪心轨迹差异; 决策表对未见 cell type 回退 R,G,B 顺序。

beam 宽度对照 (2026-08-04 晚): 将 beam 从 1 放宽到 2 (同样叠加决策层与 early-stop, 8 电路) 后, 8/8 电路仍全部成功且 final WNS 与 beam=1 完全一致 (s27 -0.21、s382 -0.93、s420 -1.75、s641/s713 -1.85、s820 -1.36、s832 -1.12、s953 -1.38), 但候选 STA 合计从 27 增至 39 (+44%)——early-stop 在两种 beam 下都在首个严格改善候选处停止, 更宽的 beam 只产生被丢弃的额外实测。该结果把“反馈学习压缩搜索宽度”的结论收紧为: **beam=1 + 反馈 + 决策层 + early-stop** 即达到该基准集上的效率最优配置。

在线自适应决策层 v2 (2026-08-05): v1 决策表是离线归纳的静态 per-cell-type 优先级表, 一旦冻结无法适应目标电路。为验证决策层的自适应能力, 我们把内环排序升级为在线 UCB1 + 指数衰减 ($\gamma = 0.98$): 每次实测 trial 后按 $\text{accept_rate} + \alpha \sqrt{\log(n)/n_k}$ 更新策略分数, 未试臂给固定 0.5 探索分 (避免乐观先验把 Buffer 顶到首位)。在完全不使用静态表、仅靠 R/G/B 冷启动回退的对照实验中, 8/8 电路全部修复成功, 且 final WNS 与静态表同等或更好:

关键证据: 在线层从零开始即收敛到静态表同等质量 (7/8), 且在 s820 上探索到静态表遗漏的修复 (`G and2_0→and2_4`, -1.28 vs -1.36), 证明修复质量不依赖人工归纳先验。代价是无先验冷启动的探索开销 (8 电路合计 STA 88 vs 27), 这恰好量化了静态表的价值 (压缩探索); 两者互补: 先以历史 trial 或静态表 warm-start, 再在线微调, 兼顾质量与效率。

表 X
IDEAL VS SPEF 寄生参数感知 STA (同一修复候选)

电路	period	ideal baseline	ideal repaired	SPEF baseline	SPEF repaired
s382	0.5 ns	-0.94	-0.93 (+0.01)	-2.37	-2.37 (0.00)
b18	13.15 ns	-0.69	-0.56 (+0.13)	-6.63	-6.63 (0.00)

表 XI
局限性

ID	limitation	处理
L31-01	SKY130 Liberty cell model 解析	已解决: Yosys miter+SAT (assign cells.v), 8/8 等价证明 SUCCESS
L31-02	8-case 全组合、无时序路径	ISCAS89 顺序电路已拓展 (period 0.5ns 制造违例)
L31-04	failure_recovery 曾为 single-iteration proxy	已解决: 真实重综合网表 + 外环成功路径打通 + 权重进 min-cut + 真实 OpenSTA WNS 接入: 4/4 电路真实改善、beam $\times$ 反馈消融证明 F4 反馈价值 (见IV)
补充	METH-12 candidate-specific timing gain 为 Stage A proxy	已解决: 内环 OpenSTA per-candidate 实测 + 外环真实 WNS evaluator (RealWnsEvaluator) 闭环

F. 寄生参数感知验证 (SPEF)

为量化 pre-layout 理想线网与真实物理负载的差距 (评审短板 1), 我们在不改变修复算法的前提下, 用 OpenSTA 的 SPEF 读取能力补充寄生参数感知验证: 将映射网表中每条 net 按估计长度 (默认 40 $\mu\text{m}/\text{unit}$) 生成简化 SPEF (含 lumped RC), 再对同一份 baseline / repaired 网表分别跑 ideal 与 SPEF 两种 STA。

真实数据 (experiments/20260805_parasitic_{s382,b18}/A-only):

结论: ** 在理想网络下有效的修复 (s382 +0.01、b18 +0.13) 在加入寄生 RC 后改善随线长连续衰减直至归零 **——这量化了 pre-layout $\rightarrow$ post-layout 的收益保持边界, 也是物理感知内环 (§ 方法) 存在的原因: FAECO 的定位是物理 ECO 的前端逻辑筛选器 (Logic Filter), 其输出为后续物理尺寸调整保留真实可测的改善余量 (b18 JOINT 在 ideal 下 +0.13、轻负载 SPEF 下 +0.11), 而不是声称 pre-layout 修复直接保证 post-layout 收敛。该 SPEF 为简化估计 (lumped RC、扇出/深度可调、非真实 P&R 版图), 完整 OpenROAD + PDK LEF/DEF 流程未接入 (见 limitation L31-01)。

G. 局限性与边界

H. ITC-99 跨基准泛化验证

为验证决策层与外环真实 WNS 闭环的跨基准泛化, 我们把 ISCAS89 上归纳的同一套流程 (外环 F1-F5 反馈 + cell-type 决策层优先级表 + beam=1 + early-stop + 真实 OpenSTA, period 0.5 ns) 直接迁移到 ITC-99 基准族。ITC-99 的.blif 使用无时钟四字段锁存 (.latch dq 0), Yosys 0.9 的 read_blif 会转出无法回读的 \$ff/\$lut 单元; 转换器在 yosys 会话内先 techmap (\$lut $\rightarrow$ \$_MUX_ 三元赋值、\$ff $\rightarrow$ \$_FF_), 再规范化模块名、加全局时钟端口并将 \$_FF_ 改写为行为级 dff, 同时修复了 b14/b15/b17 上 \$shr 桶形移位展开导致的 32 位 Yosys 内存崩溃 (std::bad_alloc)。b18/b19 发行版.blif 截断 (仅剩锁存声明、逻辑表丢失), 我们由上游完整.bench 门级文件恢复并转换。

表 XII 汇总 19 个可映射电路的修复结果 (b18/b19 因 Yosys 0.9 32 位 read_verilog 内存限制无法映射, 如实排除): 17/19 电路 WNS 真实改善 (89.5%), 包括 6 个大电路 b14/b15/b17/b20/b21/b22 全部成功; b01/b02 诚实失败 (关键路径单元无 R 等价候选且 G sizing 全部有害)。R 策略机理跨电路重现: b10/b15 的 nor4bb $\rightarrow$ and4bb 与 b11 的 lpflow_inputisolp $\rightarrow$ or2 与 s382 的 R 修复机理一致——同功能不同库单元的延迟差在 pre-layout 无 wire load 可借力时比 G sizing 更稳。19 电路外环合计 151 次候选 STA (决策层 + early-stop 生效), 小电路 4 并行约 45s、大电路 2 并行 5-6 分钟。

随后接入 64 位 Yosys 工具链 (WSL2 Ubuntu 0.33, 后统一升级为 OSS-CAD Suite nightly 0.67+146), 将此原因 32 位内存限制无法映射的 b18/b19 大电路补充: b18 (37.6 万门) 0.5ns 协议下基线 -13.34ns、修复 -13.27; b19 (75.5 万门) 基线 -17.44、修复 -16.0 (接受 R clkinv₁ $\rightarrow$ bufinv₈, 单候选改善 1.44 ns)。按 95% 原始关键路径设置现实周期 (b18 13.15 ns、b19 17.04 ns) 后, WNS 落在实际 ECO 区间: b18 基线 -0.69, 经联合修复 (JOINT, 一次 STA 同时升级 4 个关键门 nand3b₁ $\rightarrow$ 2 与 maj3₁ $\rightarrow$ 2) 改善至 -0.56 (TNS -28.18 $\rightarrow$ -22.23), 优于单门 G 的 -0.62——在 pre-layout 理想网络下, 单门 upsizing 常因输入电容拖累前级而被拒绝, 多门联合增强的净收益为正; b19 -0.90 $\rightarrow$ +0.54 达成时序收敛 (MET、TNS 清零, R 候选)。b18 剩余边界 (关键路径 36/38 为 XOR/MAJ/XNOR 复杂门, 库中无等价替换

表 XII

ITC-99 跨基准泛化: 19 个可映射电路真实 WNS 修复 (PERIOD 0.5ns, 外环反馈 + 决策层 + BEAM=1+EARLY-STOP)

电路	基线 WNS	final WNS	接受修复	STA
b01	-0.59	-0.59	— (无 R 等价候选, G 有害)	44
b02	-0.21	-0.21	— (同上)	38
b03	-1.99	-1.84	G o41ai ₁ → ₂	3
b04	-2.15	-2.10	G xnor2 ₁ → ₂	3
b05	-3.83	-3.81	G nor3b ₁ → ₂	3
b06	-0.52	-0.51	G o22ai ₁ → ₂	1
b07	-2.34	-2.33	G o21ai ₀ → ₁	1
b08	-1.29	-1.26	G and2 ₀ → ₁	4
b09	-1.12	-1.11	G nor4 ₁ → ₂	8
b10	-1.42	-1.18	R nor4bb→and4bb	3
b11	-2.68	-2.67	R lpflow_inputiso1p→or2	3
b12	-2.68	-2.42	G nor4 ₁ → ₂	3
b13	-0.92	-0.89	G o21a ₁ → ₂	7
b14	-12.23	-12.19	G or4 ₁ → ₄	5
b15	-12.35	-12.30	R nor4bb→and4bb	3
b17	-16.10	-15.62	G nor4b ₁ → ₂	4
b20	-11.20	-11.17	G and3b ₁ → ₄	5
b21	-11.29	-11.27	G and3b ₁ → ₄	5
b22	-12.08	-12.06	G mux2 ₁ → ₂	1

候选) 如实记录。

I. 工具链统一复验 (OSS-CAD 0.67)

评审缺陷 4 要求弃用 32 位 Yosys 0.9、统一工具链。FAECO 现已统一到 OSS-CAD Suite nightly (Yosys 0.67+146, 64 位): `run_yosys_mapping` 默认使用 OSS-CAD 绝对路径的 `yosys.exe` (规避 Windows CreateProcess 用父进程 PATH 解析裸命令导致的版本回退), WSL 0.33 保留为显式回退选项 (`--yosys-wsl`)。统一后关键复验:

- **s382 全修复复跑:** 真 0.67 映射下 baseline WNS -0.98, 外环多轮迭代 (候选 + 权重细化 + `random_cut`) 接受关键路径覆盖修复, final WNS -0.85——优于旧工具链 (0.33 下 -0.92)。
- **大电路映射兼容:** b18 (37.6 万门) 0.67 映射 227s 成功 (4.87 MB 网表)、b19 (75.5 万门) 510s 成功 (10.3 MB 网表), 无 `std::bad_alloc`; 配合超大锥分治切割 (§ 方法) 支撑大电路修复。
- **全量回归:** 252 项测试全绿 (含双目标联合割、物理门控、分治切割新增测试)。

J. PicoRV32 异构基准 (OOD 泛化)

评审缺陷 3 指出 ISCAS89→ITC-99 同源同构不足以证明跨设计泛化。为此引入完全异构的现代开源设计 PicoRV32 族 (同一 RTL 源的三个子模块, 结构差

表 XIII

PicoRV32 异构基准 OOD 泛化 (统一 OSS-CAD 0.67 映射 + OPENSTA 实测, PERIOD 见各电路)

电路	结构类型	DFF	baseline WNS	final WNS
picorv32	控制密集 (RISC-V 核, 7797 cells)	1597	-9.96	-8.81
picorv32_pcp_i_mul	数据密集 (32×32 乘法器)	255	-4.22	-4.11
picorv32_regs	存储密集 (寄存器堆)	992	-0.19*	-0.19*

*picorv32_regs 在 0.67 下寄存器堆映射为 enable-flop 网表、内部 reg-to-reg 路径为空 (I/O 主导), FAECO 的 reg-to-reg 修复环无候选空间; 0.33 下实测 -0.19 亦无改善 (mux2 主导、无 R 等价候选)。两种工具链均为诚实负结果。

异极大), 并在统一 OSS-CAD 0.67 工具链下直接复用 ISCAS89 上归纳的同一套流程 (双目标联合割 + 决策层 + 实测接受), 结果如实记录:

接受修复: picorv32 为 **R** 策略 (`clkinv_1`→`bufinv_16`, 单候选改善 1.13 ns)——与 s382/b11 的 R 机理一致 (同功能不同库单元延迟差在 pre-layout 下比 G 更稳); `pcpi_mul` 为 **G** 策略 (`nand3_1`→`nand3_2`, +0.05 ns, 乘法器关键路径为 nor/nand 链, upsize 收益有限)。OOD 结论 (诚实、不夸大): 3 个异构电路中控制密集与数据密集子模块真实改善, 存储密集为结构边界负结果 (与 ITC-99 b01/b02 同类: 关键单元无 R 候选、G 无净收益)——跨设计泛化不是“零损失”, 而是“流程可迁移 + 结构相关收益差异”, 决策层的在线自适应 (§IX) 在未见设计上通过前 50 轮探索收敛到与静态表相当的修复质量。

K. Hold 时间违例修复

评审的第三个关注点是场景扩展: 除 setup 违例外, 时序 ECO 还需处理 hold 违例。pre-layout 理想网络基准没有天然 hold 违例 (min slack 均匀为正), 因此注入受控 hold 场景 (`set_clock_uncertainty -hold 0.8ns`), 并让 B (buffer insertion) 策略修复最差 min (hold) 路径: 对最差 hold 端点 DFF 的 D 输入插入 buffer 链增加路径延迟, 每个候选仍由 OpenSTA 实测, 只接受**严格改善 worst min slack 且不劣化 setup WNS** 的修复——与 setup 循环使用同一 failure-aware 接受规则。修复逐端点迭代: 每轮修完当前最差端点后更新网表并重新测量。

s382 上迭代修复把 worst min slack 从 -0.36 抬到 -0.33, 且 setup WNS 未劣化反而略好 (-0.94 → -0.93), 说明 B 策略在受控 hold 场景下有真实可测收益; s27 上插 buffer 后 min slack 不动, 因该电路多端点

表 XIV

HOLD 场景扩展 (SYNTHETIC, SET_CLOCK_UNCERTAINTY -HOLD 0.8ns): B
策略迭代修复, 真实 OPENSTA 实测

电路	hold baseline	修复后 min_slack	setup WNS	接受修复
s382	-0.36	-0.33	-0.94 → -0.93	2 轮: DFF_2/DFF_3
s27	-0.36	-0.36 (无改善)	-0.28	— (4 候选全被拒绝)

并列最差 min slack, 单点修复抬不动 worst, 4 个候选全部被严格接受规则拒绝——这同时验证了 failure-aware 接受规则能阻止无效修复插入。需要透明说明的是, 这是 synthetic 场景 (理想网络无天然 hold 违例), 真实 post-layout hold 修复还需时钟树与 P&R 数据 (见 §IV-F 与 limitation)。

V. 结论

FAECO 把“重综合辅助 patch replacement”形式化为 failure-aware 的切割、替换与物理感知验证流水线: 外环以 F1–F6 失败分类驱动确定性权重调整并在公开基准上端到端跑通, 双目标联合割把等价性约束作为硬约束、关键路径覆盖割作为首轮默认候选, 从根源上消除 F1/F4 互斥; 内环以 R/G/B 混合修复经 OpenSTA 实测逐候选接受, 并以 ideal 改善 > 10ps + SPEF 复测的物理门控把物理负载从“事后验尸”变为迭代反馈信号 (F6); 对超大违例锥按逻辑深度分治切割。实验在 ISCAS89 顺序电路上 8/8 改善 WNS, 在 ITC-99 基准族上 19 个可映射电路 17/19 真实 WNS 改善 (含 b18/b19 大电路), 并在完全异构的 PicoRV32 族上验证分布外泛化。全部实验基于 EPFL v2025.1 / ISCAS89 / ITC-99 / PicoRV32 公开基准与 SKY130 HD 标准单元库, 工具链统一为 OSS-CAD Suite nightly (Yosys 0.67+), 产物可复现。

当前定位与 limitation (透明声明): FAECO 当前作为物理 ECO 的前端逻辑筛选器 (Logic Filter)——其输出的逻辑优化候选集为后续物理尺寸调整保留了可测的改善余量 (b18 JOINT 在 ideal 下 +0.13、轻负载 SPEF 下 +0.11), 而非声称 pre-layout 修复直接保证 post-layout 收敛。(1) 内环物理感知基于简化 SPEF (lumped RC、扇出/深度可调惩罚), 完整 P&R 版图验证 (OpenROAD + PDK LEF/DEF) 尚未接入; (2) 多 corner 与多工艺库仍未验证; (3) Z3 candidate/boundary 形式回验的 8-case 端到端受 mapped.v 门级实例化限制, wrapper 单元测试层面已验证; (4) 论文表格中早期实验 (Stage A/B、ISCAS89、

ITC-99 19 电路) 使用当时归档的 Yosys 0.9/0.33 版本, 统一 OSS-CAD 0.67 后已完成关键电路复验 (§IV-I、§IV-J), 全量 34 电路表格重跑作为后续工作。

未来工作: (1) 在统一 OSS-CAD 0.67 工具链下重跑全部 34 个电路表格, 并扩展 b18/b19 的多 corner 与多工艺库验证; (2) 接入 OpenROAD + PDK LEF/DEF 做迭代式物理感知闭环——第 1 轮逻辑修复生成 SPEF、发现物理负载失败则标记 F6、第 2 轮排除高扇出路径, 量化“逻辑候选 → 物理收敛”的端到端收益; (3) Z3 AIG→SMT 路径处理 mapped.v 门级实例化; (4) 结合关键路径覆盖候选与多策略内环, 量化“反馈学习 vs 宽度探索”在更大搜索空间的效率边界。

参考文献

- [1] Y.-C. Ho, C.-C. Chang, and S.-J. Chen, “ECO timing optimization using spare cells and technology remapping,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 29, no. 10, pp. 1603–1614, 2010.
- [2] Y.-C. Ho, C.-C. Chang, and S.-J. Chen, “TRECO: Dynamic technology remapping for timing engineering change orders,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 31, no. 11, pp. 1740–1752, 2012.
- [3] V. N. Kravets, “Comprehensive search for ECO rectification using symbolic sampling,” in *Proc. DAC*, 2019.
- [4] Y.-T. Chen and others, “Timing ECO optimization via Bezier curve smoothing and fixability identification,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, 2012.
- [5] Y.-T. Chen and others, “Timing ECO optimization using metal-configurable gate-array spare cells,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, 2012.
- [6] Wang et al., “ECO timing optimization with negotiation-based re-routing and logic re-structuring using spare cells,” *IEEE Trans. CAD*, 2012.
- [7] Wang et al., “Timing ECO optimization with logic restructuring,” 2016.
- [8] Y.-C. Ho et al., “ECO-map technology remapping for post-mask ECO using simulated annealing,” in *Proc. ISPD*, 2008.
- [9] J.-H. Jiang et al., “A robust functional ECO engine by SAT proof minimization and interpolation techniques,” in *Proc. ICCAD*, 2010.
- [10] C.-W. Lin et al., “Simultaneous functional and timing ECO,” in *Proc. ICCAD*, 2011.
- [11] C.-W. Lin et al., “Multi-patch generation for multi-error logic rectification,” in *Proc. ICCAD*, 2012.
- [12] Y.-L. Li et al., “Intuitive ECO synthesis for high performance circuits,” in *Proc. ICCAD*, 2013.
- [13] Jiang et al., “Resource-aware functional ECO patch generation,” *IEEE Trans. CAD*, 2016.
- [14] Luo et al., “Unified approach for simultaneous functional and timing ECO,” in *Proc. ICCAD*, 2016.
- [15] Zhuo et al., “Efficient computation of ECO patch functions,” in *Proc. DAC*, 2018.

- [16] A. Mishchenko et al., “SAT sweeping with local observability don’t-cares,” in *Proc. DAC*, 2006.
- [17] Zhong et al., “A semi-tensor product based circuit simulation for SAT-sweeping,” *IEEE Trans. CAD*, 2024.
- [18] Wang et al., “Toward exhaustive sequential redundancy removal,” *IEEE Trans. CAD*, 2024.
- [19] H. Liang et al., “O(mn) time algorithm for optimal buffer insertion of nets with m sinks,” *IEEE Trans. CAD*, 2012.
- [20] Liu et al., “RL-sizer: VLSI gate sizing for timing optimization using deep reinforcement learning,” in *Proc. ICCAD*, 2021.
- [21] Huang et al., “Learning-driven physically aware large-scale circuit gate sizing,” 2025.
- [22] Chen et al., “AiTO: Simultaneous gate sizing and buffer insertion for timing optimization with GNNs and RL,” in *Proc. DAC*, 2024.
- [23] Li et al., “Recursive learning-based virtual buffering for analytical global placement,” in *Proc. MLCAD*, 2025.
- [24] Zhang et al., “BUFFALO: PPA-configurable LLM-based buffer tree generation,” in *Proc. ICCAD*, 2025.
- [25] Zhang et al., “Restructure-tolerant timing prediction via multimodal fusion,” in *Proc. DAC*, 2023.
- [26] Zhang et al., “Disentangle, align and generalize: Learning a timing predictor from different technology nodes,” in *Proc. DAC*, 2024.
- [27] R. Brayton and A. Mishchenko, “ABC: An academic industrial-strength verification tool,” in *Proc. CAV*, 2010.
- [28] C. Wolf and J. Glaser, “Yosys – a free Verilog synthesis suite,” in *Austrochip*, 2013.
- [29] OpenROAD Project, “OpenSTA,” <https://github.com/parallaxsw/OpenSTA>, 2024.